



Amazon Simple Notification Service (SNS)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Pub/Sub

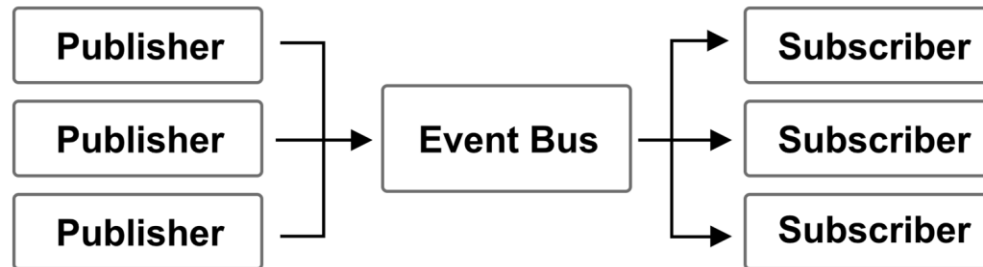
What is Pub/Sub?

Publish–subscribe pattern commonly implemented in **messaging systems**.

In a pub/sub system the sender of messages (**publishers**) does not send their messages directly to receivers. They instead send their messages to an **event bus**. The event bus categorizes their messages into groups.

Then receivers of messages (**subscribers**) subscribe to these groups.

Whenever new messages appear within their subscription the messages are immediately delivered to them.



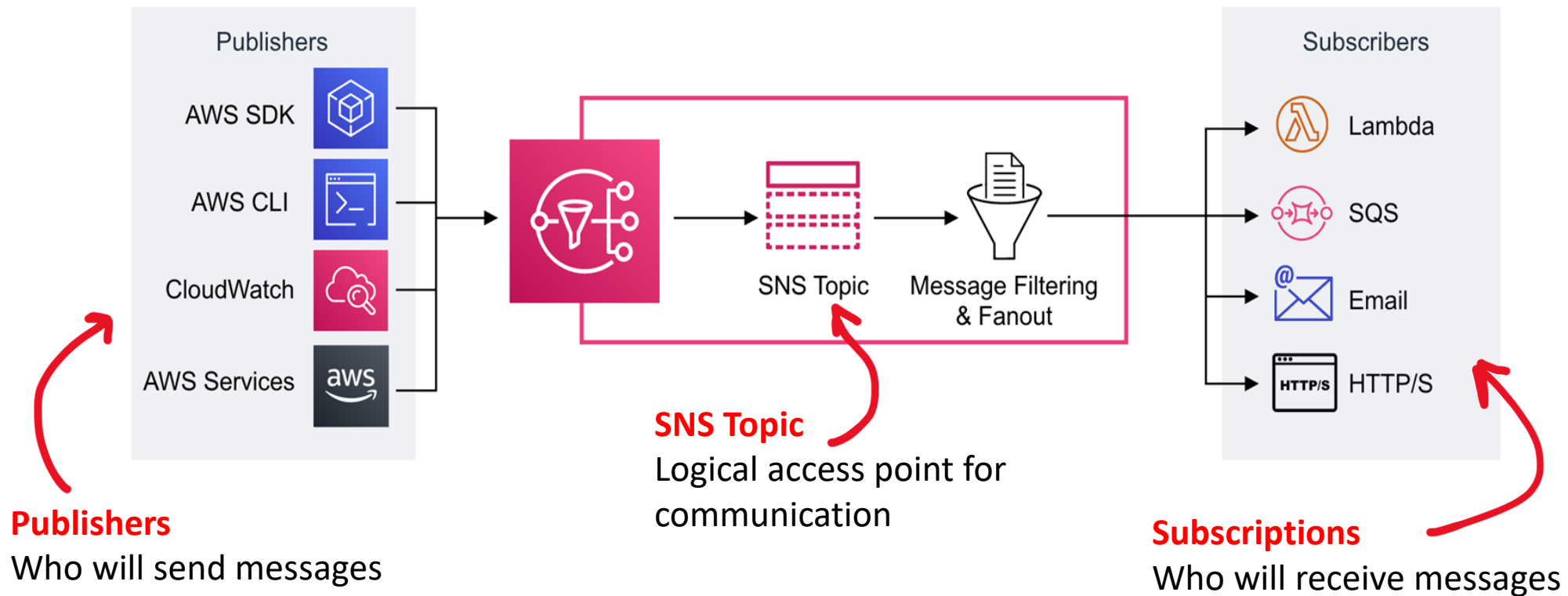
Use Case: a real-time chat system. A web-hook system

- Publishers have no knowledge of who their subscribers are.
- Subscribers do **not pull** for messages.
- Messages are instead automatically and immediately **pushed** to subscribers.
- Messages and events are interchangeable terms in pub/sub

Introduction to SNS



Simple Notification Service (SNS) is a highly available, durable, secure, fully managed **pub/sub messaging** service that enables you to **decouple** microservices, distributed systems, and serverless applications.



SNS – Sources

AWS services that can publish events to Amazon SNS topics (standard and **FIFO**).

Most services support standard topics, while some can publish to FIFO topics if message ordering and deduplication are required.

- SNS allows services to send **event notifications** to subscribers (email, Lambda, SQS, HTTPS, etc.).
- FIFO topics require message group ID and deduplication settings.
- Not all publishers support FIFO topics.

SNS – Sources

Common AWS Services That Can Publish to SNS

Compute & Application Integration

- AWS Lambda
- Amazon EC2 Auto Scaling
- Amazon ECS
- AWS Step Functions
- Amazon EventBridge
- AWS Elastic Beanstalk
- AWS App Runner
- **Amazon API Gateway**
- **Amazon Lightsail** (via CloudWatch alarms)

IoT & Mobile

- AWS IoT Core
- AWS IoT Events
- AWS IoT Greengrass
- AWS IoT Device Defender

Developer Tools

- AWS CodeBuild
- AWS CodeCommit
- AWS CodeDeploy
- AWS CodePipeline
- AWS CodeStar
- **Amazon CloudWatch Events**
- AWS CloudFormation
- AWS CloudTrail

Management & Governance

- AWS CloudWatch
- AWS Config
- AWS License Manager
- AWS Control Tower
- AWS Systems Manager
- AWS Service Catalog

Storage & Databases

- Amazon S3
- Amazon RDS
- Amazon DynamoDB
- Amazon EFS
- Amazon S3 Glacier
- Amazon ElasticCache
- Amazon Neptune

Networking & Content Delivery

- Amazon VPC
- Amazon Route 53
- Elastic Load Balancing
- AWS Global Accelerator
- AWS Direct Connect

Machine Learning & Analytics

- Amazon SageMaker
- Amazon Rekognition
- AWS Athena
- Amazon Redshift

SNS – Sources

Common AWS Services That Can Publish to SNS

Migration & Hybrid

- AWS Snowball
- AWS Application Discovery Service

End User Computing & Communication

- Amazon Chime
- Amazon WorkMail (via Lambda trigger)

Other Notables

- Amazon Pinpoint
- Amazon SES
- AWS Chatbot
- Amazon GameLift
- AWS Backup
- AWS Directory Service
- AWS Elastic Transcoder (deprecated)

Updated Notes

- **AWS Elastic Transcoder** is now largely replaced by AWS Elemental MediaConvert
- **Amazon Lightsail** has limited SNS integration and typically relies on alarms from **CloudWatch**
- Most event-driven and monitoring-based services can trigger SNS via **CloudWatch** or **EventBridge**

SNS – Destinations

Destinations are the subscribers who can receive messages.

Application-to-Application (A2A)

supports subscribers from AWS Services or web-applications

- Data Firehose
- Lambda Functions
- SQS Queue
- HTTP/S endpoint
- AWS Event Fork Pipelines

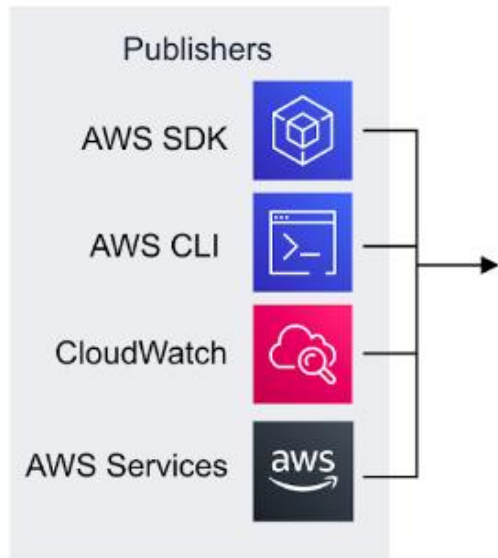
Application-to-Person (A2P)

Supports subscribers from human-entities:

- Mobile applications
- Mobile phone numbers (SMS aka text messaging)
- Email Address
- AWS Chatbot
- PagerDuty

SNS - Topics

Publishers don't care about the subscribers' protocol



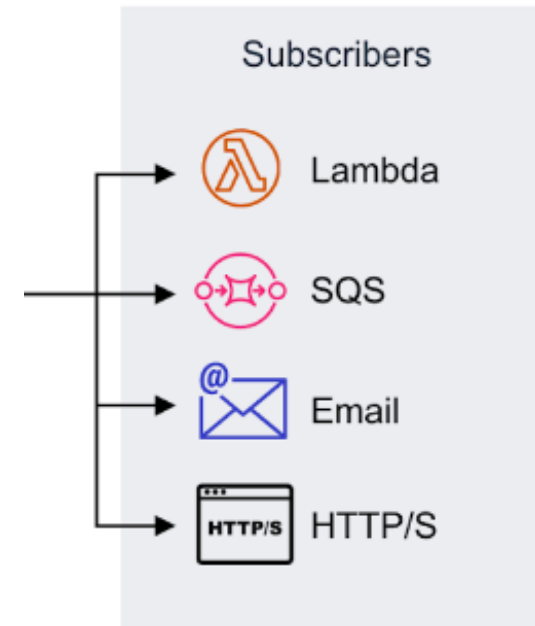
Topics allow you to group multiple subscriptions together.

A topic is able to deliver to multiple protocols at once eg. email, text message, http/s

When topics deliver messages to subscribers, it will automatically format your message according to the subscriber's chosen protocol

You can encrypt Topics via **KMS**

Subscribers listen for incoming messages



```
aws sns create-topic --name my-topic
aws sns create-topic --name my-topic.fifo --attributes FifoTopic=true
```

SNS – Topics

There are two types of SNS Topics.

- Standard
- FIFO (First in First Out) – guarantees the order of the messages

```
aws sns create-topic --name my-topic  
aws sns create-topic --name my-topic.fifo --attributes FifoTopic=true
```



When creating a fifo topic **it has to have “.fifo”** added to the end of the topic name.

SNS - Topics

	Standard	First-in-First-Out
Throughput	High throughput	Lower throughput compared to Standard
Delivery	At least once (possibility of duplicates)	Exactly once (no duplicates)
Ordering	No ordering guarantees	Messages are delivered in the order they are sent within a message group
Use-Case	where the volume of messages is high and exact ordering/delivery isn't critical: • alerts • notifications	where the order and exact delivery are crucial • banking transactions • ordered data processing
Message Grouping	N/A	Supports message grouping, allowing multiple ordered streams within the same topic

SNS – Publishing Messages

The most common way to work with SNS is by having your application **programmatically publish messages to the SNS Topic**.

```
require 'aws-sdk-sns'

# Initialize the SNS client
sns_client = Aws::SNS::Client.new

# Specify your SNS topic ARN
topic_arn = 'arn:aws:sns:ca-central-1:123456789012:my-topic'

# The message you want to send
message = "Hello World!"

# Publish the message
begin
  resp = sns_client.publish({
    topic_arn: topic_arn,
    message: message
  })
  puts "Message sent successfully! Message ID: #{resp.message_id}"
rescue Aws::SNS::Errors::ServiceError => error
  puts "Error sending message: #{error}"
end
```

Ruby example

SNS – Publishing Large Messages

To publish large messages to SNS you can use the the libraries:

- Amazon SNS Extended Client Library for Java/Python

<https://github.com/aws-labs/amazon-sns-python-extended-client-lib>

These libraries are useful for messages that are larger than the current maximum of 256 KB, **with a maximum of 2 GB**.

Both libraries save the actual payload to an **S3 bucket**, and publish the reference of the stored S3 object to the SNS topic.

```
import boto3
import sns_extended_client

sns = boto3.client('sns')
sns.large_payload_support = 'bucket-name'

# boto SNS.Topic resource
resource = boto3.resource('sns')
topic = resource.Topic('topic-arn')
platform_endpoint = resource.PlatformEndpoint('endpoint-arn')
platform_endpoint.large_payload_support = 'my-bucket-name'

# If you want to keep it always enabled.
platform_endpoint.always_through_s3 = True

# publish the large message
sns.publish(
    Message="message",
    MessageAttributes={
        "S3Key": {
            "DataType": "String",
            "StringValue": "--S3--Key--",
        }
    },
)
```

SNS – Message Attributes

SNS supports delivery of **message attributes**, which let **you provide structured metadata items about the message**

The supported data types for attributes:

- String
- String.Array
- Number
- Binary

```
aws sns publish \  
--topic-arn "arn:aws:sns:region:account-id:topic-name" \  
--message "Notification with various data types for attributes" \  
--message-attributes message-attributes.json
```

```
{"OrderNumber": {  
  "DataType": "number",  
  "NumberValue": 4152141,  
}  
...}
```



SNS – Batch Publishing

BatchPublish action can **publish upto 10 messages at a time**



Sending messages in batches can help you reduce the SNS costs by a factor of 10

```
require 'aws-sdk-sns'

sns_client = Aws::SNS::Client.new(region: 'us-west-2')

topic_arn = 'arn:aws:sns:us-west-2:123456789012:my_topic'

messages = [
  {id: "msg1",
   message: "First message",
   message_attributes: {
     "AttributeKey1" => {data_type: "String", string_value: "AttributeValue1"}
   }
},
  {id: "msg2",
   message: "Second message",
   message_attributes: {
     "AttributeKey2" => {data_type: "Number", string_value: "123"}
   }
}
]

sns_client.publish_batch({
  topic_arn: topic_arn,
  publish_batch_request_entries: messages
})
```

SNS – Message Structure

If you want to send different messages to different kinds of subscribers
set message structure to json and supply a json file as the message

```
aws sns publish --topic-arn "arn:aws:sns:region:account-id:topic-name" \  
--message-structure json \  
--message file://message.json
```

```
{  
  "default": "Default message",  
  "email": "Email message",  
  "sms": "SMS message",  
  "sqs": "SQS message",  
  "http": "HTTP message",  
  "https": "HTTPS message",  
  "application": "Application message"  
}
```

SNS - Subscriptions

Subscriptions (1)				
EditDeleteRequest confirmationConfirm subscriptionCreate subscription				
Search				
< 1 > ⚙				
ID	Endpoint	Status	Protocol	
☐ 33d3e87b-413e-412a-bca3-32b2b14cb24d	andrew@exampro.co	✔ Confirmed	EMAIL	

To receive messages from a topic, you need to **create a Subscription.**

A subscription can only subscribe to one protocol and one topic.

Protocol
The type of endpoint to subscribe

Select protocol ▼

- HTTP
- HTTPS
- Email
- Email-JSON
- Amazon SQS
- AWS Lambda
- Platform application endpoint
- SMS

The following protocols:

- **HTTP and HTTPS** create webhooks to your web application
- **Email** good for internal email notifications (only supports **plain text**)
- **Email-JSON** sends you JSON via email
- **Amazon SQS** place SNS message into SQS queue
- **AWS Lambda** triggers a lambda function
- **SMS** sends a text message
- **Platform application endpoints** Mobile Push

SNS – Filter Policy

SNS Filter Policy allows you to **filter a subset a messages only to be delivery.**

FilterPolicyScope

- MessageAttributes – filter based on message attributes
- MessageBody — filter base on message body

Filtering Options

- AND logic
- OR logic
- OR operator
- Key matching
- Numeric value exact matching
- Numeric value anything-but matching
- Numeric value range matching
- String value exact matching
- String value anything-but matching
- String matching using a prefix with the anything-but operator
- String value equals-ignore case
- String value IP address matching
- String value prefix matching
- String value suffix matching

For MessageBody you'd set **--attribute-value MessageBody**

```
aws sns set-subscription-attributes \  
--subscription-arn ... \  
--attribute-name FilterPolicy \  
--attribute-value file://policy.json
```



```
{  
  "store": ["example_corp"],  
  "event": [{"anything-but": "order_cancelled"}],  
  "customer_interests": [  
    "rugby",  
    "football",  
    "baseball"  
  ],  
  "price_usd": [{"numeric": [">=", 100]}]  
}
```

SNS – Message Data Protection

Message data protection **safeguards the data that's published to your Amazon SNS topics** by using data **protection policies to audit, mask, redact, or block the sensitive information** that moves between applications or AWS services

Scans for:

- Personally identifiable information (PII)
- Protected health information (PHI)

via data identifiers

You can choose to use predefined data identifiers:

- Name
- Addresses
- Credit card numbers

You can create your own data identifiers

Supports the following actions:

- Audit — Audit upto 99% of data published than send findings to CloudWatch, S3 or Data Firehose
- De-identify — mask or redact data
- Deny — Block data from being sent



Message data protection can help to reduce financial, legal, and regulatory risks by complying with privacy regulations such as HIPAA, GDPR, PCI, and FedRAMP.

Message data protection is only supported for standard SNS Topics

SNS – Message Data Protection

```
aws sns put-data-protection-policy \  
--resource-arn arn:aws:sns:us-east-1:123456789012:mytopic \  
--data-protection-policy file:./policy.json
```

Example data protection policy that
will **mask credit card numbers** with
the pound symbol

```
{  
  "Name": "__example_data_protection_policy",  
  "Description": "Example data protection policy",  
  "Version": "2021-06-01",  
  "Statement": [  
    {  
      "DataDirection": "Inbound",  
      "Principal": [  
        "arn:aws:iam::123456789012:user/ExampleUser"  
      ],  
      "DataIdentifier": [  
        "arn:aws:dataprotection::aws:data-identifier/CreditCardNumber"  
      ],  
      "Operation": {  
        "Deidentify": {  
          "MaskConfig": {  
            "MaskWithCharacter": "#"  
          }  
        }  
      }  
    }  
  ]  
}
```



SNS – Raw Message Delivery

Raw Message Delivery avoid having Amazon Data Firehose, Amazon SQS, and HTTP/S endpoints process the JSON formatting of messages.

```
aws sns set-subscription-attributes \  
--subscription-arn ... \  
--attribute-name RawMessageDelivery \  
--attribute-value true
```



Data Firehose and SQS: metadata is stripped from the published message and the message is sent as is.

HTTP/S Endpoint: HTTP header **x-amz-sns-rawdelivery** with its value set to true, indicating the message should not be formatted

SNS – Delivery Policy

SNS Delivery Policy defined **how SNS retries the delivery of messages** when server-side errors occur
Each delivery protocol has its own delivery policy.



When the delivery policy is exhausted, SNS stops retrying the delivery and discards the message unless a dead-letter queue is attached to the subscription.

	Data Firehose	Lambda	SQS	SMTP	SNS	Mobile Push
Immediate Retry Phase (No Delay)	3 times, without delay			0 times, without delay		
Pre-Backoff Phase	2 times, 1 second apart			2 times, 10 seconds apart		
Backoff Phase	10 times, with exponential backoff, from 1 second to 20 seconds			10 times, with exponential backoff, from 10 seconds to 600 seconds (10 minutes)		
Post-Backoff Phase	100,000 times, 20 seconds apart			38 times, 600 seconds (10 minutes) apart		

These data policies cannot be changed with the exception of HTTP/S

SNS – Delivery Policy

Setting your own delivery policy
for HTTP/s endpoint

There are four options for backup function

- arithmetic
- exponential
- geometric
- linear

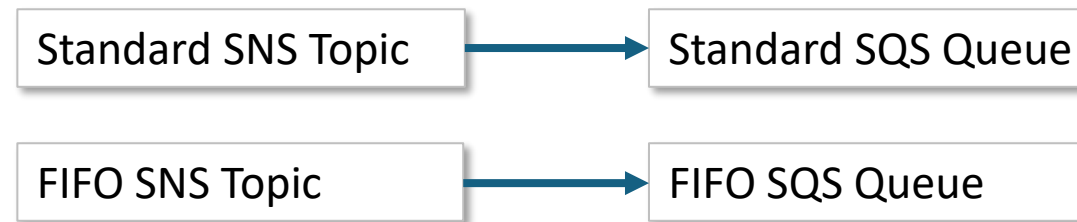
```
aws sns set-subscription-attributes \  
--subscription-arn ... \  
--attribute-name DeliveryPolicy \  
--attribute-value file://retry-policy.json
```

```
{  
  "healthyRetryPolicy": {  
    "minDelayTarget": 1,  
    "maxDelayTarget": 60,  
    "numRetries": 50,  
    "numNoDelayRetries": 3,  
    "numMinDelayRetries": 2,  
    "numMaxDelayRetries": 35,  
    "backoffFunction": "exponential"  
  },  
  "throttlePolicy": {  
    "maxReceivesPerSecond": 10  
  },  
  "requestPolicy": {  
    "headerContentType": "application/json"  
  }  
}
```

SNS – Dead Letter Queue

SNS Dead Letter Queue (DLQ) will **send failed message attempts to an SQS queue.**

Topic and Queue type **need to match**



Configure a SNS topic to send to a a DLQ via **the AWS CLI**

```
aws sns set-subscription-attributes \  
--subscription-arn ... \  
--attribute-name RedrivePolicy \  
--attribute-value deadLetterTargetArn=<SQS ARN>
```



A queue policy will need to be configured to allow SNS to send events to the SQS queue

Application As Subscriber

Send push notification messages directly to apps on **mobile devices**.

Push notification messages sent to a mobile endpoint can appear in the mobile app as message alerts, badge updates, or even sound alerts.

Protocol
The type of endpoint to subscribe

Select protocol ▼

- HTTP
- HTTPS
- Email
- Email-JSON
- Amazon SQS
- AWS Lambda
- Platform application endpoint**
- SMS

